National Vocational and Technical Training Commission

SUBMITTED BY: Suleman Ahmad

COURSE TITLE: NAVTTC AI

ROLL NO: CND-02325528

TASK: Week#6

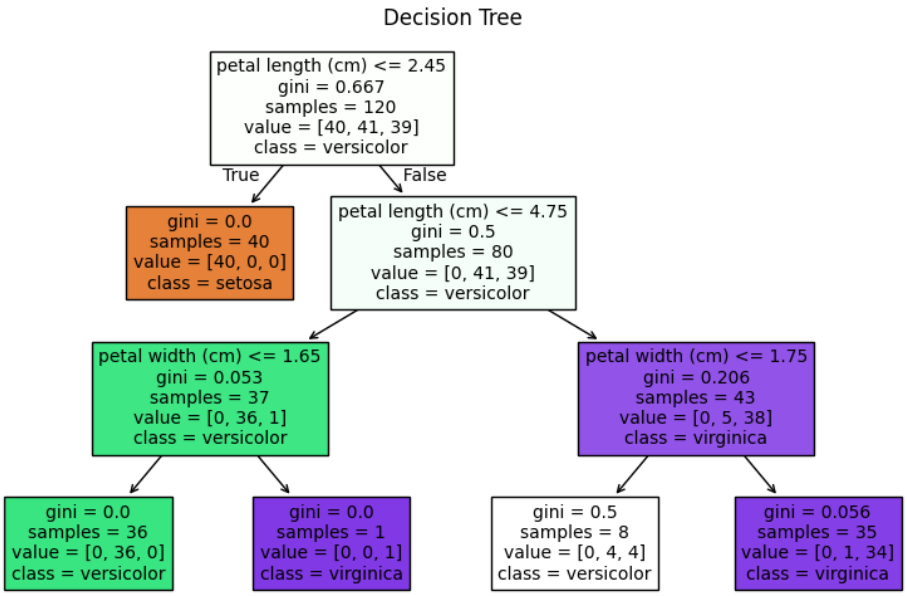
Task_53

week6_Task > t53.py > ...

```python
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create model
model = DecisionTreeClassifier(max_depth=3)
model.fit(X_train, y_train)

# Accuracy
print("Accuracy:", model.score(X_test, y_test))

# ===== ONE IMPORTANT GRAPH =====
plt.figure(figsize=(10,6))
plot_tree(model,
          feature_names=data.feature_names,
          class_names=data.target_names,
          filled=True)
plt.title("Decision Tree")
plt.show()
```

OUTPUT

```
[Running] python -u "c:\Users\B-T\Desktop\ALLaboutAI\week6\week6_Task\t53.py"
Accuracy: 1.0
```


Decision Tree
petal length (cm) <= 2.45
gini = 0.667
samples = 120
value = [40, 41, 39]
class = versicolor
True
False
gini = 0.0
samples = 40
value = [40, 0, 0]
class = setosa
petal length (cm) <= 4.75
gini = 0.5
samples = 80
value = [0, 41, 39]
class = versicolor
petal width (cm) <= 1.65
gini = 0.053
samples = 37
value = [0, 36, 1]
class = versicolor
petal width (cm) <= 1.75
gini = 0.206
samples = 43
value = [0, 5, 38]
class = virginica
gini = 0.0
samples = 36
value = [0, 36, 0]
class = versicolor
gini = 0.0
samples = 1
value = [0, 0, 1]
class = virginica
gini = 0.5
samples = 8
value = [0, 4, 4]
class = versicolor
gini = 0.056
samples = 35
value = [0, 1, 34]
class = virginica

Task_54

week6_Task > t54.py > ...

```python
# ============================================
# Support Vector Machine (SVM) - Classification
# ============================================

# 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 2. Load Dataset
csv_path = Path(__file__).resolve().parent / "bill_authentication.csv"
if not csv_path.exists():
    raise FileNotFoundError(f"CSV file not found at: {csv_path}")
df = pd.read_csv(csv_path)

print("Dataset Shape:", df.shape)
print("\nFirst 5 Rows:\n", df.head())

# 3. Data Preparation
X = df.drop('Class', axis=1)   # Features
y = df['Class']                # Target

# 4. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("\nTraining Samples:", len(X_train))
```

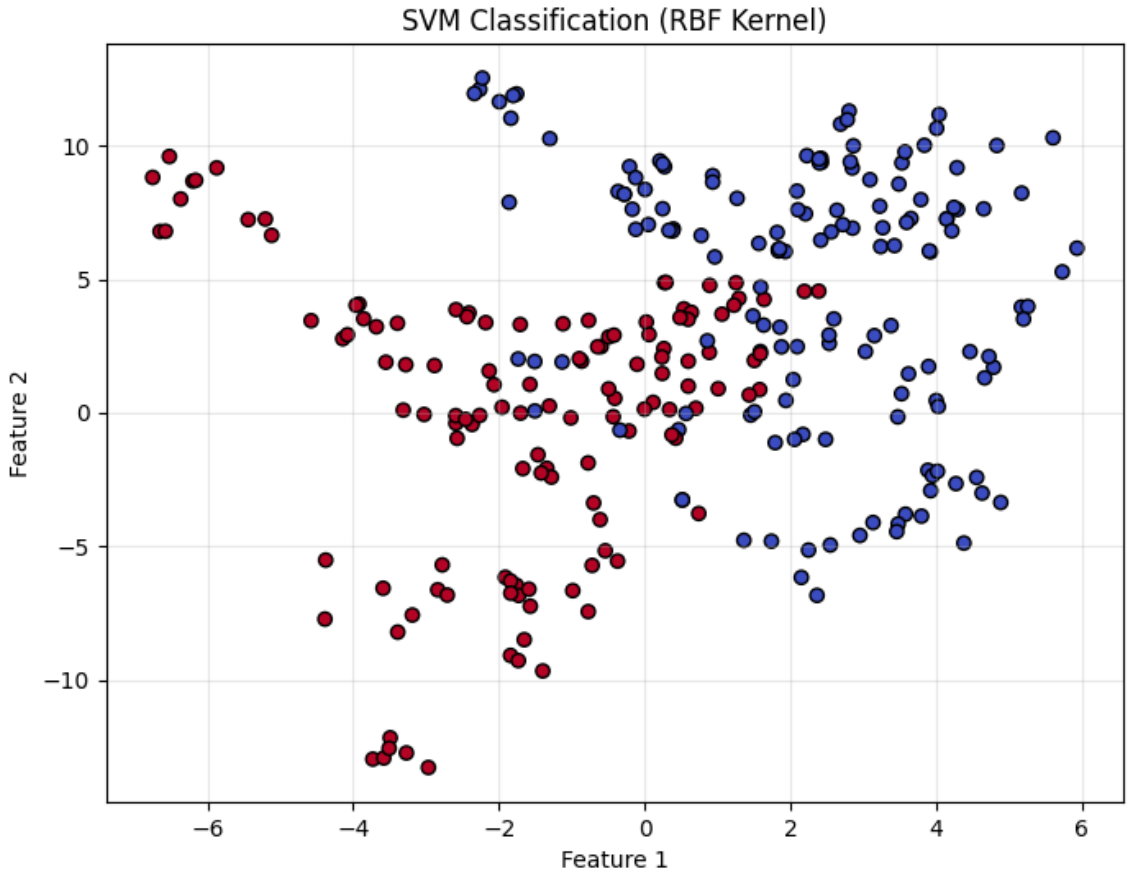
week6_Task > t54.py > ...

```python

print("\nTraining Samples:", len(X_train))
print("Testing Samples :", len(X_test))

# 5. Create SVM Model
model = SVC(kernel='rbf')   # You can change to 'linear'

# 6. Train Model
model.fit(X_train, y_train)

# 7. Predictions
y_pred = model.predict(X_test)

# 8. Evaluation
print("\n=== Model Evaluation ===")
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

# 9. Visualization (ONE GRAPH)
# Using first two features for plotting
plt.figure(figsize=(8, 6))

plt.scatter(
    X_test.iloc[:, 0],
    X_test.iloc[:, 1],
    c=y_pred,
    cmap='coolwarm',
    edgecolors='k'
```

```
64  )
65
66  plt.xlabel("Feature 1")
67  plt.ylabel("Feature 2")
68  plt.title("SVM Classification (RBF Kernel)")
69  plt.grid(True, alpha=0.3)
70
71  plt.show()
72
73  # 10. Single Prediction Example
74  sample = X_test.iloc[0:1]
75  prediction = model.predict(sample)
76
77  print("\nSample Prediction:")
78  print("Input Features:\n", sample)
79  print("Predicted Class:", prediction[0])
80
81  # ========================================
82  # End of Code
83  # ========================================
```

OUTPUT

```
Training Samples: 1097
Testing Samples : 275

=== Model Evaluation ===
Accuracy: 1.0

Confusion Matrix:
[[148   0]
 [  0 127]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       148
           1       1.00      1.00      1.00       127

    accuracy                           1.00       275
   macro avg       1.00      1.00      1.00       275
weighted avg       1.00      1.00      1.00       275
```


SVM Classification (RBF Kernel)
Feature 2
Feature 1
10
5
0
−5
−10
−6
−4
−2
0
2
4
6

TASK_55

week6_Task > 55.py > ...

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ── 1. Load Both Files ──────────────────────────────────
train = pd.read_csv(r"C:\Users\B-T\Desktop\ALLaboutAI\DataSets\DailyDelhiClimateTrain.csv",
                    parse_dates=["date"], index_col="date")

test  = pd.read_csv(r"C:\Users\B-T\Desktop\ALLaboutAI\DataSets\DailyDelhiClimateTest.csv",
                    parse_dates=["date"], index_col="date")

df = pd.concat([train, test])

print("Train shape:", train.shape, "| Date range:", train.index.min(), "->", train.index.max())
print("Test  shape:", test.shape,  "| Date range:", test.index.min(),  "->", test.index.max())
print("\nFirst 5 rows:\n", train.head())
print("\nData types:\n", train.dtypes)

# ── 2. TIME-BASED INDEXING & SLICING ────────────────────
print("\n--- Single day ---")
print(train.loc['2013-01-01'])

print("\n--- Month slice (Jan-Mar 2014) ---")
print(train.loc['2014-01':'2014-03'])

print("\n--- Full year 2015 ---")
print(train.loc['2015'])

# ── 3. RESAMPLING – Weekly & Monthly ────────────────────
fig, axes = plt.subplots(2, 1, figsize=(12, 7))
```

week6_Task > 55.py > ...

```python

train['meantemp'].resample('W').mean().plot(ax=axes[0], color='steelblue')
axes[0].set_title('Weekly Mean Temperature (Train)')
axes[0].set_ylabel('Temp (C)')

train['meantemp'].resample('ME').mean().plot(ax=axes[1], color='tomato')
axes[1].set_title('Monthly Mean Temperature (Train)')
axes[1].set_ylabel('Temp (C)')

plt.tight_layout()
plt.show()

# ── 4. SEASONALITY – Boxplot by Month & Weekday ─────────
train['month']   = train.index.month
train['weekday'] = train.index.day_name()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.boxplot(x='month', y='meantemp', data=train, ax=axes[0], palette='coolwarm')
axes[0].set_title('Seasonality by Month')
axes[0].set_xlabel('Month')
axes[0].set_ylabel('Mean Temp (C)')

weekday_order = ['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday']
sns.boxplot(x='weekday', y='meantemp', data=train,
            order=weekday_order, ax=axes[1], palette='Set2')
axes[1].set_title('Seasonality by Weekday')
axes[1].tick_params(axis='x', rotation=30)

plt.tight_layout()
plt.show()
```

```
plt.tight_layout()
plt.show()

# ── 5. ROLLING WINDOWS & TRENDS ──────────────────────────
plt.figure(figsize=(12, 5))
train['meantemp'].plot(alpha=0.3, label='Daily', color='gray')
train['meantemp'].rolling(window=7).mean().plot(label='7-Day Rolling', color='steelblue')
train['meantemp'].rolling(window=365).mean().plot(label='365-Day Trend', color='tomato', linewidth=2)
plt.title('Rolling Windows & Long-Term Trend')
plt.ylabel('Temp (C)')
plt.legend()
plt.tight_layout()
plt.show()

# ── 6. TRAIN vs TEST ─────────────────────────────────────
plt.figure(figsize=(12, 4))
train['meantemp'].plot(label='Train (2013-2017)', color='steelblue')
test['meantemp'].plot(label='Test (2017)', color='tomato')
plt.title('Train vs Test - Mean Temperature')
plt.ylabel('Temp (C)')
plt.legend()
plt.tight_layout()
plt.show()
```

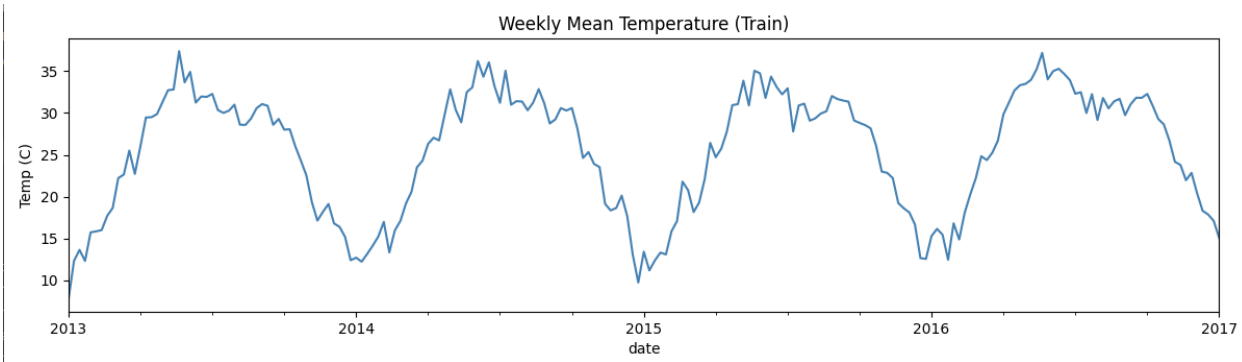
OUTPUT

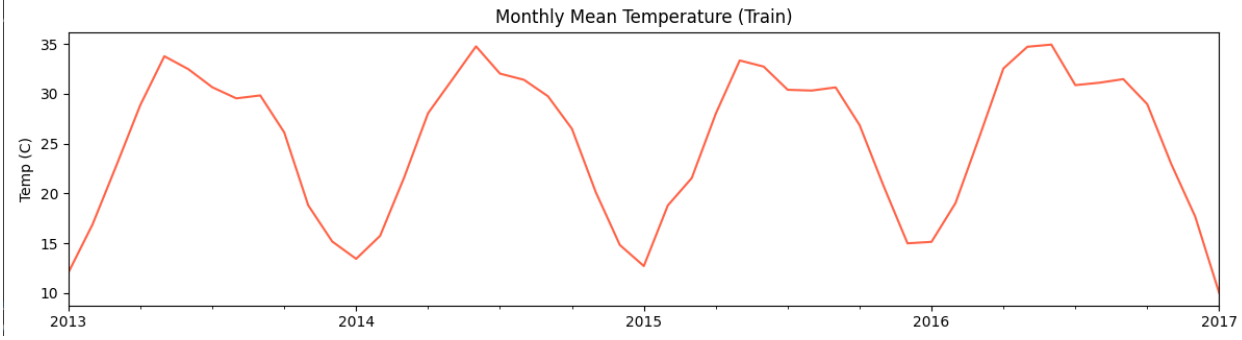
```
[Running] python -u "c:\Users\B-T\Desktop\ALLaboutAI\week6\week6_Task\55.py"
Train shape: (1462, 4) | Date range: 2013-01-01 00:00:00 -> 2017-01-01 00:00:00
Test  shape: (114, 4) | Date range: 2017-01-01 00:00:00 -> 2017-04-24 00:00:00

First 5 rows:
            meantemp   humidity  wind_speed  meanpressure
date
2013-01-01  10.000000  84.500000    0.000000   1015.666667
2013-01-02   7.400000  92.000000    2.980000   1017.800000
2013-01-03   7.166667  87.000000    4.633333   1018.666667
2013-01-04   8.666667  71.333333    1.233333   1017.166667
2013-01-05   6.000000  86.833333    3.700000   1016.500000

Data types:
meantemp        float64
humidity        float64
wind_speed      float64
meanpressure    float64
dtype: object
```

```
--- Single day ---
meantemp          10.000000
humidity          84.500000
wind_speed         0.000000
meanpressure    1015.666667
Name: 2013-01-01 00:00:00, dtype: float64

--- Month slice (Jan-Mar 2014) ---
            meantemp   humidity  wind_speed  meanpressure
date
2014-01-01  13.375000  89.625000      7.6500   1021.000000
2014-01-02  11.000000  78.375000      8.1000   1020.250000
2014-01-03  12.500000  74.875000      5.3250   1017.750000
2014-01-04  12.875000  88.125000      1.1625   1016.250000
2014-01-05  12.375000  89.000000      0.4625   1014.500000
...               ...        ...         ...          ...
2014-03-27  23.500000  63.250000      4.8875   1011.750000
2014-03-28  24.428571  67.571429      4.5000   1011.142857
2014-03-29  25.875000  55.625000      8.3500   1011.250000
2014-03-30  24.125000  46.750000     13.9000   1009.875000
2014-03-31  24.125000  44.125000     13.2000   1008.250000

[90 rows x 4 columns]
```

```
--- Full year 2015 ---
            meantemp  humidity  wind_speed  meanpressure
date
2015-01-01    14.750    72.000      0.9250      1017.500
2015-01-02    14.875    96.625      3.0125      1017.875
2015-01-03    15.125    92.000      0.9250      1017.375
2015-01-04    14.125    78.750      9.5125      1019.625
2015-01-05    14.000    69.375     15.0500      1016.000
...              ...       ...         ...           ...
2015-12-27    15.375    63.250      7.8875      1020.625
2015-12-28    17.125    58.125     10.8875      1020.875
2015-12-29    16.375    65.000      7.4125      1018.125
2015-12-30    15.500    71.750      2.1000      1017.500
2015-12-31    15.000    71.375      2.0875      1020.500

[365 rows x 4 columns]
c:\Users\B-T\Desktop\ALLaboutAI\week6\week6_Task\55.py:49: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x='month', y='meantemp', data=train, ax=axes[0], palette='coolwarm')
c:\Users\B-T\Desktop\ALLaboutAI\week6\week6_Task\55.py:55: FutureWarning:
```

```
  sns.boxplot(x='month', y='meantemp', data=train, ax=axes[0], palette='coolwarm')
c:\Users\B-T\Desktop\ALLaboutAI\week6\week6_Task\55.py:55: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x='weekday', y='meantemp', data=train,
```


Weekly Mean Temperature (Train)
Temp (C)
35
30
25
20
15
10
2013
2014
2015
2016
2017
date


Monthly Mean Temperature (Train)
Temp (C)
35
30
25
20
15
10
2013
2014
2015
2016
2017

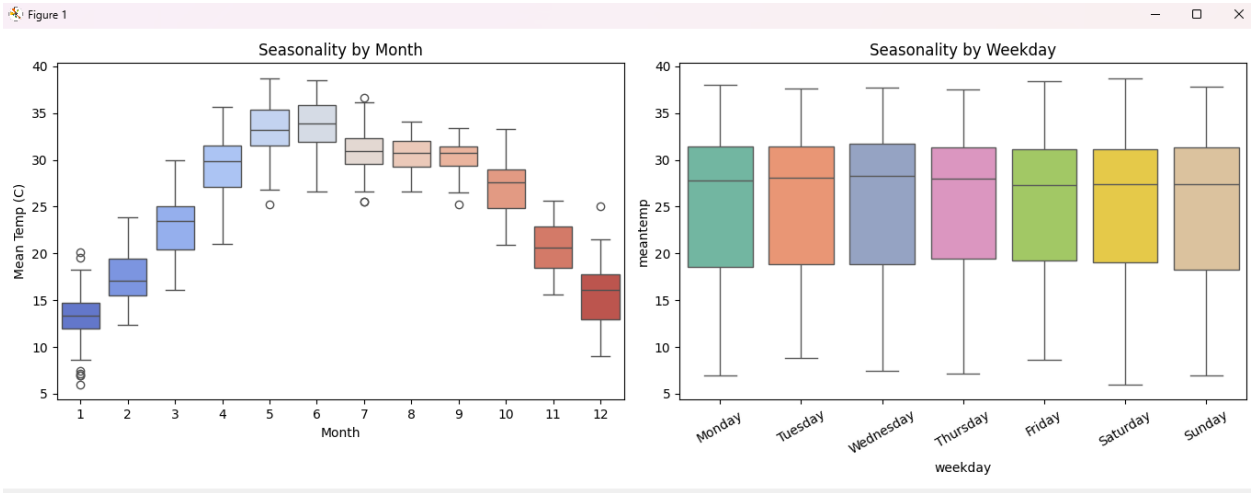
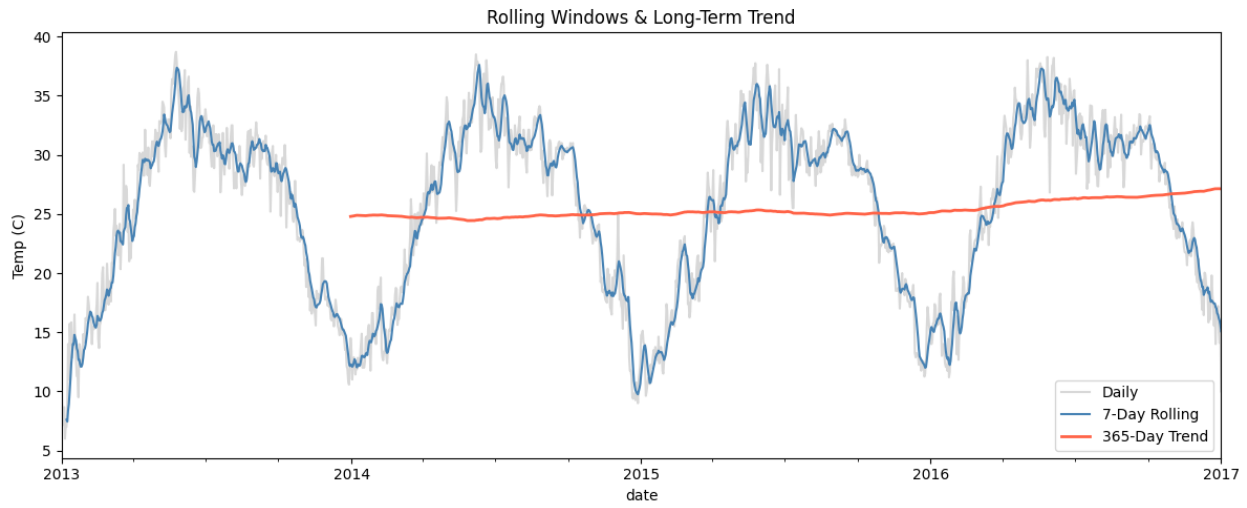
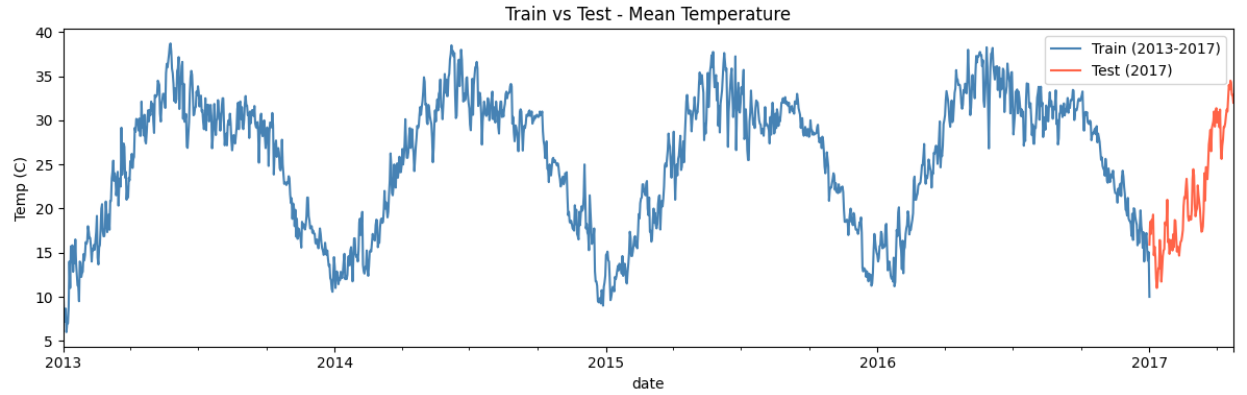


Figure 1
Seasonality by Month
Mean Temp (C)
40
35
30
25
20
15
10
5
1
2
3
4
5
6
7
8
9
10
11
12
Month
Seasonality by Weekday
meantemp
40
35
30
25
20
15
10
5
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday
weekday


Rolling Windows & Long-Term Trend
Temp (C)
date
Daily
7-Day Rolling
365-Day Trend
2013
2014
2015
2016
2017


Train vs Test - Mean Temperature
Temp (C)
date
Train (2013-2017)
Test (2017)
2013
2014
2015
2016
2017